



**R.D. & S.H. NATIONAL COLLEGE & S.W.A. SCIENCE
COLLEGE**

Bandra, Mumbai-400050

**Department of Computer Science &
Data Science**

Soft Computing

**M.Sc. Data Science Part I –Semester
II JOURNAL 2025-2026**

Course Code: - PSDSP512

Seat No. :



R.D. & S.H. NATIONAL COLLEGE & S.W.A. SCIENCE COLLEGE

Bandra, Mumbai-400050.



Department of Computer Science & Data Science

CERTIFICATE

This is to certify that **Mr. Molla Zamir Hossain** of MSc. Part I (Sem II) class has satisfactorily completed **10/10** Practical in the subject of **Soft Computing** as a part of M.Sc. Degree Course in Data Science during the academic year 2025– 2026.

Faculty In-Charge

**Co-Ordinator
Department of Computer
Science**

External Examiner

INDEX

Sr. No.	Title	Date	Page No.	Sign
1	Write a program to implement logical gates AND, OR and NOT with McCulloch-Pitts.		1	
2	Write a program to implement Hebb's rule.		2	
3	Implement Kohonen Self organizing map.		3-4	
4	Solve the Hamming network given the exemplar vectors.		5	
5	Write a program for implementing BAM network.		6-7	
6	Implement a program to find the winning neuron using MaxNet.		8	
7	Implement De-Morgan's Law.		9	
8	Implement Union, Intersection, Complement and Difference operations on fuzzy sets.		10-11	
9	Create fuzzy relation by Cartesian product of any two fuzzy sets		12	
10	Perform max-min composition on any two fuzzy relations.		13-14	

Practical 1

Aim: Write a program to implement logical gates AND, OR and NOT, with McCulloch-Pitts.

Code:

```
def mcp_neuron(inputs, weights, threshold):
    weighted_sum = sum(i * w for i, w in zip(inputs, weights))
    return 1 if weighted_sum >= threshold else 0
x = [0, 1]
w = [-1, -1]
t = 0

def AND(x1, x2):
    return mcp_neuron([x1, x2], [1, 1], 2)

def OR(x1, x2):
    return mcp_neuron([x1, x2], [1, 1], 1)

def NOT(x):
    return mcp_neuron([x], [-1], 0)

print("AND Gate")
for x1 in [0, 1]:
    for x2 in [0, 1]:
        print(x1, x2, "->", AND(x1, x2))

print("\nOR Gate")
for x1 in [0, 1]:
    for x2 in [0, 1]:
        print(x1, x2, "->", OR(x1, x2))

print("\nNOT Gate")
for x in [0, 1]:
    print(x, "->", NOT(x))
```

Output:

```
AND Gate
0 0 -> 0
0 1 -> 0
1 0 -> 0
1 1 -> 1
```

```
OR Gate
```

Practical 2

Aim: Write a program to implement Hebb's learning rule.

Code:

```
# Hebbian Learning Function
def hebbian_learning(inputs, targets, learning_rate=1.0):
    # Number of features
    n_features = inputs.shape[1]

    # Initialize weights to zero
    weights = np.zeros(n_features)

    # Training
    for i in range(len(inputs)):
        x = inputs[i]
        y = targets[i]

    return weights + learning_rate * x * y # Hebbian update

# Example dataset (AND logic)
inputs = np.array([
    [0, 0],
    [0, 1],
    [1, 0],
    [1, 1]
])

targets = np.array([0, 0, 0, 1])

# Convert 0 → -1 (common in Hebbian learning)
inputs = np.where(inputs == 0, -1, 1)
targets = np.where(targets == 0, -1, 1)

# Train
weights = hebbian_learning(inputs, targets)

print("Final Weights:", weights)
```

Output:

Final Weights: [2. 2.]

Practical 3

Aim: To implement Kohonen Self-Organizing Maps Code.

Code:

```
import numpy as np
```

```
class SOM:
```

```
    def __init__(self, x, y, input_dim, learning_rate=0.5, sigma=None, iterations=1000):
```

```
        self.x = x
```

```
        self.y = y
```

```
        self.input_dim = input_dim
```

```
        self.learning_rate = learning_rate
```

```
        self.iterations = iterations
```

```
        if sigma is None:
```

```
            sigma = max(x, y) / 2.0
```

```
        self.sigma = sigma
```

```
        # Initialize weight vectors randomly
```

```
        self.weights = np.random.rand(x, y, input_dim)
```

```
        # Coordinate grid for neurons
```

```
        self._init_neuron_locations()
```

```
    def _init_neuron_locations(self):
```

```
        self.neuron_locations = np.array(
```

```
            [[i, j] for i in range(self.x) for j in range(self.y)]
```

```
        )
```

```
    def _euclidean_distance(self, x, w):
```

```
        return np.linalg.norm(x - w, axis=-1)
```

```
    def _find_bmu(self, sample):
```

```
        distances = np.linalg.norm(self.weights - sample, axis=2)
```

```
        bmu_index = np.unravel_index(np.argmin(distances), (self.x, self.y))
```

```
        return bmu_index
```

```
    def _decay_learning_rate(self, t):
```

```
        return self.learning_rate * np.exp(-t / self.iterations)
```

```
    def _decay_sigma(self, t):
```

```
        return self.sigma * np.exp(-t / self.iterations)
```

```
    def _neighborhood(self, bmu, sigma):
```

```
        d = np.sum((self.neuron_locations - np.array(bmu))**2, axis=1)
```

```
        return np.exp(-d / (2 * (sigma ** 2))).reshape(self.x, self.y)
```

```
    def train(self, data):
```

```
        for t in range(self.iterations):
```

```
            sample = data[np.random.randint(0, len(data))]
```

```

        bmu = self._find_bmu(sample)

        lr = self._decay_learning_rate(t)
        sigma = self._decay_sigma(t)

        neighborhood = self._neighborhood(bmu, sigma)

        # Update weights
        for i in range(self.x):
            for j in range(self.y):
                influence = neighborhood[i, j]
                self.weights[i, j] += lr * influence * (sample - self.weights[i, j])

    def map_vects(self, data):
        mapping = []
        for sample in data:
            mapping.append(self._find_bmu(sample))
        return mapping

# Sample dataset (2D)
data = np.array([
    [0.1, 0.2],
    [0.3, 0.2],
    [0.8, 0.7],
    [0.9, 0.6],
    [0.4, 0.5]
])

# Create SOM (5x5 grid, input dimension = 2)
som = SOM(x=5, y=5, input_dim=2, learning_rate=0.5, iterations=2000)

# Train SOM
som.train(data)

# Map data to BMUs
mapped = som.map_vects(data)

print("BMU mapping:")
for i, m in enumerate(mapped):
    print(f"Data {data[i]} -> BMU {m}")

```

Output:

```

BMU mapping:
Data [0.1 0.2] -> BMU (np.int64(4), np.int64(0))
Data [0.3 0.2] -> BMU (np.int64(0), np.int64(0))
Data [0.8 0.7] -> BMU (np.int64(2), np.int64(4))
Data [0.9 0.6] -> BMU (np.int64(0), np.int64(3))
Data [0.4 0.5] -> BMU (np.int64(3), np.int64(2))

```

Practical 4

Aim: Solve the Hamming Network, given the exemplar vectors.

Code:

```
import numpy as np

# Define the exemplar vectors (pre-defined inputs)
exemplar_vectors = np.array([
    [1, 0, 1, 0, 1, 1, 0, 1],
    [0, 1, 0, 1, 0, 0, 1, 0],
    [1, 1, 1, 1, 0, 1, 0, 0]
])

# Define the input vector
input_vector = np.array([1, 0, 1, 1, 0, 1, 0, 1])

def hamming_distance(v1, v2):
    """
    Compute the Hamming distance between two binary vectors.
    """
    return np.sum(v1 != v2)

def hamming_network(input_vector, exemplar_vectors):
    """
    Find the exemplar vector with the smallest Hamming distance to the input vector.
    """
    distances = np.array([hamming_distance(input_vector, ev) for ev in exemplar_vectors])
    min_distance_index = np.argmin(distances)
    return min_distance_index, distances[min_distance_index]

# Run the Hamming Network
index, distance = hamming_network(input_vector, exemplar_vectors)

# Output the result
print(f"The input vector is closest to exemplar vector at index {index} with a Hamming distance of {distance}.")
```

```
↔ The input vector is closest to exemplar vector at index 0 with a Hamming distance of 2.
```


Practical 5

Aim: Write a program for implementing BAM network.

Code:

```
import numpy as np

# Defining BAM class
class BAM:
    def __init__(self):
        self.weights = None

    def train(self, patterns_A, patterns_B):
        """
        Train the BAM using Hebbian learning rule.
        Each pair of patterns (A, B) is used to update the weight matrix.
        """
        num_features_A = patterns_A.shape[1]
        num_features_B = patterns_B.shape[1]
        self.weights = np.zeros((num_features_A, num_features_B))

        for a, b in zip(patterns_A, patterns_B):
            self.weights += np.outer(a, b)

    def recall_A(self, pattern_B):
        """
        Recall pattern A given pattern B.
        """
        result = np.dot(pattern_B, self.weights.T)
        return np.sign(result)

    def recall_B(self, pattern_A):
        """
        Recall pattern B given pattern A.
        """
        result = np.dot(pattern_A, self.weights)
        return np.sign(result)

# Example usage
if __name__ == "__main__":
    # Define the training patterns
    patterns_A = np.array([[1, 1, -1], [-1, 1, 1], [-1, -1, -1]])
    patterns_B = np.array([[1, -1], [-1, 1], [1, 1]])

    # Initialize BAM
    bam = BAM()
```

```
# Train BAM with the patterns
bam.train(patterns_A, patterns_B)

# Test recall for pattern B
test_pattern_B = np.array([1, -1])
recalled_pattern_A = bam.recall_A(test_pattern_B)
print("Recalled Pattern A for test pattern B", test_pattern_B, "is:", recalled_pattern_A)

# Test recall for pattern A
test_pattern_A = np.array([1, 1, -1])
recalled_pattern_B = bam.recall_B(test_pattern_A)
print("Recalled Pattern B for test pattern A", test_pattern_A, "is:", recalled_pattern_B)
```

```
⇒ Recalled Pattern A for test pattern B [ 1 -1] is: [ 1.  0. -1.]
   Recalled Pattern B for test pattern A [ 1  1 -1] is: [ 1. -1.]
```

Practical 6

Aim: Implement a program to find the winning neuron using MaxNet.

Code:

```
import numpy as np
def maxnet(input_vector, epsilon=0.1, max_iterations=100):
    # Initialize the activations
    activations = np.copy(input_vector)

    for _ in range(max_iterations):
        # Compute the inhibition for each neuron
        inhibition = epsilon * (np.sum(activations) - activations)

        # Update activations
        new_activations = activations - inhibition

        # Set negative activations to zero
        new_activations[new_activations < 0] = 0

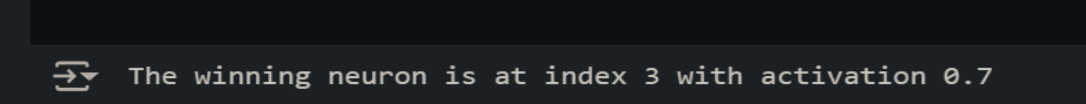
        # Check if only one neuron is active
        if np.count_nonzero(new_activations) == 1:
            break

        # Prepare for next iteration
        activations = new_activations

    # Return the index of the winning neuron
    return np.argmax(activations)

# Example usage:
input_vector = np.array([0.2, 0.5, 0.1, 0.7, 0.4])
winning_neuron = maxnet(input_vector)
print(f"The winning neuron is at index {winning_neuron} with activation {input_vector[winning_neuron]}")
```

Output:



```
➞ The winning neuron is at index 3 with activation 0.7
```

Practical 7

Aim: Implement De-Morgan's Law

Code:

```
def de_morgans_law_1(A, B):
    # Law 1:  $\sim(A \vee B) = \sim A \wedge \sim B$ 
    not_A_or_B = not (A or B)
    not_A_and_not_B = (not A) and (not B)
    return not_A_or_B, not_A_and_not_B

def de_morgans_law_2(A, B):
    # Law 2:  $\sim(A \wedge B) = \sim A \vee \sim B$ 
    not_A_and_B = not (A and B)
    not_A_or_not_B = (not A) or (not B)
    return not_A_and_B, not_A_or_not_B

# Get user input for A and B once
A_input = input("Enter A (True/False): ").strip().lower()
B_input = input("Enter B (True/False): ").strip().lower()

# Convert to boolean
A = A_input == "true"
B = B_input == "true"

# Apply Law 1
result_1 = de_morgans_law_1(A, B)
print("\nDe Morgan's Law 1:  $\sim(A \vee B) = \sim A \wedge \sim B$ ")
print(f' $\sim(\{A\} \vee \{B\}) = \{result\_1[0]\}$ ')
print(f' $\sim\{A\} \wedge \sim\{B\} = \{result\_1[1]\}$ ')
print(f'Law holds:  $\{result\_1[0]\} == \{result\_1[1]\}$ ')

# Apply Law 2
result_2 = de_morgans_law_2(A, B)
print("\nDe Morgan's Law 2:  $\sim(A \wedge B) = \sim A \vee \sim B$ ")
print(f' $\sim(\{A\} \wedge \{B\}) = \{result\_2[0]\}$ ')
print(f' $\sim\{A\} \vee \sim\{B\} = \{result\_2[1]\}$ ')
print(f'Law holds:  $\{result\_2[0]\} == \{result\_2[1]\}$ ')

```

```
Enter A (True/False): True
Enter B (True/False): False

De Morgan's Law 1:  $\sim(A \vee B) = \sim A \wedge \sim B$ 
 $\sim(\text{True} \vee \text{False}) = \text{False}$ 
 $\sim\text{True} \wedge \sim\text{False} = \text{False}$ 
Law holds: True

De Morgan's Law 2:  $\sim(A \wedge B) = \sim A \vee \sim B$ 
 $\sim(\text{True} \wedge \text{False}) = \text{True}$ 
 $\sim\text{True} \vee \sim\text{False} = \text{True}$ 
Law holds: True

```

Practical 8

Aim: Implement Union, Intersection, Complement, and Difference operations, on fuzzy sets

Code:

```
# Implementing union operation of Fuzzy sets
def fuzzy_union(A, B):
    """Compute the union of two fuzzy sets A and B. Union:  $\max(A(x), B(x))$ """
    return {x: max(A.get(x, 0), B.get(x, 0)) for x in set(A).union(B)}

# Implementing intersection operation of Fuzzy sets
def fuzzy_intersection(A, B):
    """Compute the intersection of two fuzzy sets A and B. Intersection:  $\min(A(x), B(x))$ """
    return {x: min(A.get(x, 0), B.get(x, 0)) for x in set(A).intersection(B)}

# Implementing complement operation of Fuzzy sets
def fuzzy_complement(A):
    """Compute the complement of a fuzzy set A. Complement:  $1 - A(x)$ """
    return {x: 1 - A[x] for x in A}

# Implementing difference operation of Fuzzy sets
def fuzzy_difference(A, B):
    """Compute the difference of two fuzzy sets A and B. Difference:  $\min(A(x), 1 - B(x))$ """
    return {x: min(A.get(x, 0), 1 - B.get(x, 0)) for x in set(A).union(B)}

# Example fuzzy sets
A = {'x1': 0.1, 'x2': 0.4, 'x3': 0.7}
B = {'x2': 0.5, 'x3': 0.2, 'x4': 0.8}

# Perform fuzzy set operations
union_result = fuzzy_union(A, B)
intersection_result = fuzzy_intersection(A, B)
complement_result_A = fuzzy_complement(A)
difference_result = fuzzy_difference(A, B)

# Print the results
print("Fuzzy Set A:", A)
print("Fuzzy Set B:", B)

print("\nResults of operations performed on the fuzzy sets:")
print("Union ( $A \cup B$ ):", union_result)
print("Intersection ( $A \cap B$ ):", intersection_result)
print("Complement (A):", complement_result_A)
print("Difference ( $A - B$ ):", difference_result)
```

```
Fuzzy Set A: {'x1': 0.1, 'x2': 0.4, 'x3': 0.7}
```

```
Fuzzy Set B: {'x2': 0.5, 'x3': 0.2, 'x4': 0.8}
```

```
Results of operations performed on the fuzzy sets:
```

```
Union (A  $\cup$  B): {'x3': 0.7, 'x1': 0.1, 'x2': 0.5, 'x4': 0.8}
```

```
Intersection (A  $\cap$  B): {'x3': 0.2, 'x2': 0.4}
```

```
Complement (A'): {'x1': 0.9, 'x2': 0.6, 'x3': 0.30000000000000004}
```

```
Difference (A - B): {'x3': 0.7, 'x1': 0.1, 'x2': 0.4, 'x4': 0}
```

Practical 9

Aim: Create Fuzzy relation by Cartesian product of any two fuzzy sets.

Code:

```
# Define a function to implement Cartesian product
def cartesian_product_fuzzy_relation(A, B):
    """
    Create a fuzzy relation by Cartesian product of fuzzy sets A and B.
    The membership value of the pair (x, y) is min(A(x), B(y)).
    """
    relation = {}
    for x in A:
        for y in B:
            relation[(x, y)] = min(A[x], B[y])
    return relation

# Fuzzy sets input:
A = {'x1': 0.7, 'x2': 0.4, 'x3': 0.9}
B = {'y1': 0.6, 'y2': 0.8, 'y3': 0.5}

# Compute the Cartesian product fuzzy relation
relation = cartesian_product_fuzzy_relation(A, B)

# Print the results
print("Fuzzy Set A:", A)
print("Fuzzy Set B:", B)

print("\nCartesian Product Fuzzy Relation:")
for (x, y), value in relation.items():
    print(f'({x}, {y}): {value}')
```

```
Fuzzy Set A: {'x1': 0.7, 'x2': 0.4, 'x3': 0.9}
Fuzzy Set B: {'y1': 0.6, 'y2': 0.8, 'y3': 0.5}

Cartesian Product Fuzzy Relation:
(x1, y1): 0.6
(x1, y2): 0.7
(x1, y3): 0.5
(x2, y1): 0.4
(x2, y2): 0.4
(x2, y3): 0.4
(x3, y1): 0.6
(x3, y2): 0.8
(x3, y3): 0.5
```

Practical 10

Aim: Perform max-min composition on any two fuzzy relations.

Code:

```
# Cartesian product of fuzzy sets
def cartesian_product_fuzzy_relation(A, B):
    """
    Create a fuzzy relation by Cartesian product of fuzzy sets A and B.
    Membership of (x, y) = min(A[x], B[y])
    """
    relation = {}
    for x in A:
        for y in B:
            relation[(x, y)] = min(A[x], B[y])
    return relation

# Max-Min composition of fuzzy relations
def max_min_composition(R, S):
    """
    Perform max-min composition on fuzzy relations R and S.
    """
    T = {}
    x_elements = set(x for x, _ in R)
    y_elements = set(y for x, y in R)
    z_elements = set(z for y, z in S)

    for x in x_elements:
        for z in z_elements:
            min_values = []
            for y in y_elements:
                if (x, y) in R and (y, z) in S:
                    min_values.append(min(R[(x, y)], S[(y, z)]))
            if min_values:
                T[(x, z)] = max(min_values)
    return T

# Fuzzy sets
A = {'x1': 0.7, 'x2': 0.4, 'x3': 0.9}
B = {'y1': 0.6, 'y2': 0.8, 'y3': 0.5}
C = {'z1': 0.5, 'z2': 0.9, 'z3': 0.3}

# Compute fuzzy relations
R = cartesian_product_fuzzy_relation(A, B)
S = cartesian_product_fuzzy_relation(B, C)
```



```

# Max-min composition
T = max_min_composition(R, S)

# Display results
print("Fuzzy Set A:", A)
print("Fuzzy Set B:", B)
print("Fuzzy Set C:", C)

print("\nFuzzy Relation R ( $A \times B$ ):")
for (x, y), value in R.items():
    print(f'({x}, {y}): {value}')

print("\nFuzzy Relation S ( $B \times C$ ):")
for (y, z), value in S.items():
    print(f'({y}, {z}): {value}')

print("\nMax-Min Composition ( $R \circ S$ ):")
for (x, z), value in T.items():
    print(f'({x}, {z}): {value}')

```

```

Fuzzy Set A: {'x1': 0.7, 'x2': 0.4, 'x3': 0.9}
Fuzzy Set B: {'y1': 0.6, 'y2': 0.8, 'y3': 0.5}
Fuzzy Set C: {'z1': 0.5, 'z2': 0.9, 'z3': 0.3}

Fuzzy Relation R ( $A \times B$ ):
(x1, y1): 0.6
(x1, y2): 0.7
(x1, y3): 0.5
(x2, y1): 0.4
(x2, y2): 0.4
(x2, y3): 0.4
(x3, y1): 0.6
(x3, y2): 0.8
(x3, y3): 0.5

```